

Directory hygiene” [54]. We want to highlight well-known attacks relevant to our capability evaluation, organized by the attack stage they typically occur in:

Initial Access. Initially, the attacker is situated within the enterprise network but does not possess AD user account credentials thus their goal is to compromise an existing AD user account. Typical attack paths for this often include password-based attacks. Due to the existence of active countermeasures (Section 2.1.3), traditional brute-force attacks are not performed due to the risk of detection and account lock-out. Instead, *Password Spraying* attacks using few common passwords (typically less than three per user account to prevent account lock-out) or scenario-specific password lists are employed. On a network level, active Kerberos AS-REP roasting attacks exploit the combination of cryptographically weak protocols combined with a common insecure configuration to gain a user’s password hash. Similarly, passive network sniffing attacks can be performed to capture a user’s NTLM hash containing their hashed password). Password hashes are typically used with password cracking tools such as *hashcat* or *john-the-ripper* to extract the underlying plain-text passwords.

Lateral Movement and Privilege Escalation. The compromised AD user account can subsequently be used to further enumerate the Active Directory. The goal is to compromise further user or system accounts, to thus elevate the attacker’s privileges, as well as to gain *domain dominance*, i.e., gaining access to a domain administrator account. As compromised accounts are typically employed to re-perform enumeration steps, traditional waterfall-influenced attack methodologies such as Lockheed-Martin Cyber Kill Chain are often replaced with iterative methodologies such as the Mandiant Attacker Lifecycle [37].

Typical Attacks performed during this stage include Kerberoasting SPN attacks which target credentials for network services, searching network file shares for sensitive information such as user credentials, abusing overly permissive AD schema permissions, or accessing network services such as MSSQL or Exchange servers.

2.1.3 Common Defenses. Typical defenses against cyber attacks include *Network Intrusion Detection Systems* (NIDS) and host-based *Endpoint Detection and Response* (EDR) tools. The latter are the successor to traditional anti-virus (AV) and anti-malware solutions. NIDS typically are used to notify defensive personnel. As cyber attacks increasingly happen outside on-call duty times, we focus on automated EDR software.

The goal of EDR software is to detect and automatically quarantine an attacker and their employed tools such as backdoors or implants [27]. They often use a combination of heuristics, fingerprinting, and behavioral analysis to detect intruders. Automated counter-measures range from terminating processes, locking user accounts, to quarantining the whole computer system from the enterprise network.

Originally, EDR software was exclusively provided by third-party vendors but Microsoft introduced *Microsoft Defender* in 20024 as free *Microsoft AntiSpyware* add-on for Windows. It was subsequently renamed into *Microsoft Defender* and released as part of *Windows Vista*. Within Windows 7, Defender was superseded by *Microsoft Security Essentials*, a traditional AV solution. An improved version of Defender, now evolved into a full EDR solution, was enabled by default in *Windows 8* and *Windows Server 2016*, making this EDR the dominant EDR on the market.

2.1.4 Attack Taxonomy. MITRE ATT&CK is a classification of potential attacks and not, as often assumed, a testbed nor attack methodology. They use three different abstraction levels for categorizing attacks: Tactics, Techniques and Procedures (short TTPs). The 14 tactics describe the high-level goal of an attack, e.g., *Initial Access*, *Credential Access*, or *Exfiltration*. Each tactic consists of multiple potential techniques, e.g., the tactic *Credential Access* includes *T1557: Adversary-in-the-Middle*. Procedures finally give examples how an attacker could achieve a given technique.

2.2 Penetration-Testing

Penetration-Testing is a broad domain and typically describes offensive approaches to investigate the security posture of target systems. Ethical hackers typically provide a report of their findings, often consisting of detected vulnerabilities and insecure configurations, to their respective customer which in turn remediates found problems.

Currently there is a single paper by Happe and Cito [15] that investigates the different types of penetration testing assignments, their respective workflows, and problems therein. They identify different types of attacks, of which the three most relevant for this work are *Vulnerability Scans*, *Internal Network Tests*, and *Red-Teaming*. During Vulnerability Scans, the target system is typically scanned using an automated vulnerability scanner. The goal is breadth, not depth; found vulnerabilities are often not exploited but only detected. The scope is very limited, often only a single system and attacks are loud, i.e., they are easily detected by defenders.

Red-Teaming is the opposite: they target a company as a whole and often start “externally” to the main company network with social engineering attacks. A red-teaming campaign is undercover, i.e., defenders do not know that they are under attack, attacks are kept “quiet” to prevent detection. They target depth, i.e., achieving a single goal stated by the customer, not breadth, i.e., detecting all vulnerabilities in the company. Operations are typically performed manually.

In between lie Internal Network Tests, often called *Assumed Breach Simulations*. In these attacks, the attacker is placed within the local enterprise network and tries to achieve domain dominance, i.e., tries to become domain or forest administrator which is the user with the highest permissions within the target network. This is based upon the assumption that an attacker will eventually gain access to the local network (“breach the network”), and that for efficiency reasons, testing can focus upon the subsequent movements of the attacker within the network. Within these scenarios breadth is the goal, i.e., finding as many vulnerabilities as possible. To achieve this, multiple vulnerabilities must be combined into attack chains thus depth must also be explored. Assumed Breach simulations can reach from being “loud” to “quiet”.

2.2.1 Testbeds for Assumed Breach Simulations in Enterprise Networks. We investigated existing testbeds for human penetration-testers for their potential for benchmarking LLM-driven penetration-test solutions. Within Happe and Cito’s interview study [15] with professional penetration testers, interviewees mentioned Capture-the-Flag (short CTF) scenarios as good learning exercises that enable information transfer into penetration testing work assignments. CTFs typically are provided as virtual machines or hosted within the cloud, trainees can access the virtual machine and try to exploit vulnerabilities to achieve a defined target, often indicated through a “flag” which is a file containing a unique identifier that indicates that the trainee was able to achieve the task. Examples of web sites allowing for this type of training are TryHackMe³ or HackTheBox⁴.

CTF-style challenges are also commonly used for verifying penetration testers’ capabilities during industry penetration test certification exams. Within these exams, trainees are typically given access to a CTF-style testbed and are tasked to achieve tasks within a short timeframe, ranging from 8h to a week. Examples of such goals are “compromise four out of five domain controllers in the testbed” or “become domain admin”. Well-known certifications that follow this approach are OSCP⁵, OSCE⁶, CRT0⁷, CRTP⁸, among others.

³<https://tryhackme.com/>

⁴<https://www.hackthebox.com/>

⁵<https://www.offsec.com/courses/pen-200/>

⁶<https://www.offsec.com/certificates/osce3/>

⁷<https://training.zeropointsecurity.co.uk/courses/red-team-ops>

⁸<https://www.alteredsecurity.com/post/certified-red-team-professional-crtp>

2.2.2 *Costs of Penetration Testing.* [Indeed.com](https://www.indeed.com)⁹, a metasearch engine that aggregates job postings from thousands of websites and employment firms, reports the average salary of a penetration tester with \$53.09/h. Penetration Test companies typically charge between \$100–\$300/h.

2.3 LLM-aided Task-Planning

We highlight recent improvements in both intra-task solving, i.e., allowing a LLM to solve a given task, as well as within intra-task solving, i.e., allowing LLMs to split up larger tasks into smaller sub-tasks that are subsequently solved by either the LLM itself or a dedicated sub-task LLM. If applicable, we focus upon techniques used within the cybersecurity domain.

2.3.1 *Intra-Task Improvements.* The emergence of chain-of-thought (CoT) prompting has marked a significant advancement in leveraging LLMs for tasks that require complex, multi-step reasoning. Initially introduced by Wei et al. [62], CoT prompting facilitates enhanced reasoning by allowing the model to articulate intermediate steps prior to arriving at a final answer. When CoT prompting is paired with few-shot learning paradigms, it has demonstrated marked improvements in handling tasks that necessitate intricate reasoning processes. Building on this idea, subsequent work by Kojima et al. [30] introduced the concept of zero-shot CoT prompting. This approach leverages a simple yet effective modification by appending the directive “Let’s think step by step” to the prompt, thereby eliciting a structured chain of reasoning without the need for pre-crafted examples. While this technique simplifies the prompt design and can yield encouraging results in various contexts, the manual effort typically required to curate effective and diverse demonstration examples in few-shot prompting remains a hurdle, potentially leading to suboptimal solutions in more complex scenarios. Addressing these limitations, Zhang et al. [71] proposed a methodology that completely removes the need for manual example engineering. Their approach uses LLMs themselves to iteratively generate reasoning chains, each initiated by the “Let’s think step by step” prompt. This self-generative method not only reduces human intervention but also holds promise for more consistent and robust reasoning, particularly in contexts where diverse reasoning patterns are crucial.

The ReAct framework, introduced by Yao et al. [69], enables LLMs to generate reasoning traces and task-specific actions in an interleaved manner. Generating reasoning traces allows the model to manage action plans, while the action step allows for interaction with and information gathering from external sources. LLMs can interact with external tools to retrieve additional information, leading to more reliable and factual responses.

Reflexion [55] uses linguistic feedback to strengthen language-based agents. It works by converting environmental feedback into linguistic feedback (self-reflection), which is then used as context for an LLM agent in the subsequent episode. This process allows the agent to learn quickly and effectively from past mistakes, leading to improved performance on a variety of complex tasks.

2.3.2 *Reasoning LLMs.* Large Reasoning Models (LRMs or “reasoning LLMs”) are LLMs that are explicitly trained to perform native thinking or chain-of-thought [42, 64]. The availability of OpenAI’s initial *o1-preview* model [24] was announced in September 2024 [41] and it was included in OpenAI’s API in December 2024 [41]. Other examples of reasoning models are Alibaba’s QWEN3 [66] or DeepSeek’s R1 model [13].

⁹<https://www.indeed.com>

According to OpenAI, they trained their reasoning models to “*think longer and harder about complex tasks, making them effective at strategizing, planning solutions to complex problems, and making decisions based on large volumes of ambiguous information*” [44]. These models trade higher quality output for longer processing times during inference.

While evaluations attest their capabilities [72], their native inclusion of Chain-of-Thought techniques during training reduces the efficacy of prior established prompt-engineering techniques. Li et al. [33] show that manually incorporating Chain-of-Thought while using reasoning models reduces the instruction-following performance of these models. Software developer websites focusing upon LLM development [6, 47] note that few-shot prompting should be minimized or removed to avoid LLM confusion. OpenAI itself refers to prior prompt techniques as *Boomer-Prompts* [43].

Recent research questions the reasoning capabilities of LRMs. Petrov et al. [46] evaluate the performance of reasoning models against the 2025 USA Math Olympiad, esp. their capability to generate mathematical proofs. Their results show the difference between pattern-recognition and mathematical reasoning, and indicate that reasoning models only perform well if data similar to their given tasks was included in their training data. Shojaee et al. [56] investigate the performance of reasoning models in puzzle environments. They differentiate their results based on the complexity of the puzzle tasks. On easy tasks, non-reasoning models outperform reasoning models due to the latter performing “over-thinking” and creating convoluted Chain-of-Thoughts. On tasks with moderate difficulty, reasoning models’ methodical approach (CoT) let them outperform non-reasoning models. On difficult tasks, both types of LLMs failed.

Applied to our research scenario of security testing, we assume that pattern-matching is well-suited to solve penetration testing tasks as indicated by security practitioners reporting being able to apply knowledge learning during CTF-exercises to real-life penetration-testing [15]. In addition, we focus on ubiquitous Microsoft Active Directory enterprise networks of whose security vulnerabilities ample background data should be included in the LLMs’ training data set. We also assume that the difficulty of penetration-testing AD networks falls into the moderate difficulty category, thus making LRM well-suited for the task.

2.3.3 Inter-Task Planing. Wang et al. [61] introduce the generic *Plan-and-Solve* prompting pattern for tackling multi-step plans. It consists of two components: first, devising a plan to divide the entire task into smaller subtasks, and then carrying out the subtasks according to the plan. A contemporary Open-Source project, *BabyAGI*¹⁰, popularized this approach. In the cybersecurity domain, Happe and Cito [18] investigated the use of *Plan-and-Solve* for Linux privilege escalation attacks.

Deng et al. [7] use *Pentest Task Trees* (PTT) to track penetration test progress. A PTT is a hierarchical data structure that allows a LLM to both create a high-level plan for a penetration-test as well as note findings occurring during the penetration test. The task tree itself is very similar to a structured todo list, written out in Markdown. Deng et al. used CTF-style challenges to verify the efficacy of their approach.

2.4 Automated Penetration Testing

2.4.1 Traditional Automated Scanners. Penetration Testers use automated tooling during vulnerability assessments. Examples for such tooling are *nmap*, *OpenVAS*, or *Nessus*. These solutions are typically noisy and cannot be deployed during red-teaming or assumed breach simulations. Typically, they are checklist- or rule-based and perform thousands of tests during a testing run. They commonly perform enumeration but do not abuse and execute detected vulnerabilities, thus limiting both their achieved depth and breadth. For example, if a network share enumeration detects a file with potential user account credentials, they are not used for other tests.

¹⁰<https://github.com/yoheinakajima/babyagi>

MITRE Caldera [2] is often used during Purple-Teaming exercises. In Purple-Teaming, attackers work in lock-step with the defenders. In these exercises the attacker emulates the tactics, techniques and procedures of an existing advanced persistent threat actor (APT). They execute an attack and then analyze if the defenders were able to detect the attack and have chosen adequate counter-measures. To automate this process, Caldera can be configured with a set of attack techniques (e.g., SMB enumeration, Password-Spraying with a static credential list) which are then automatically executed. While this is semantically very similar to the mentioned vulnerability scanners, the selection and scope of the executed operations are strictly within the Assumed Breach scenarios. Caldera does not create a high-level penetration-testing strategy as it is manually configured by the attackers. This is by intent, as the strategy should emulate existing well-documented APTs.

2.4.2 ML for Offensive Security (Non-LLM). Partially Observable Markov Decision Processes (POMDP) [50, 51] have been investigated to automatically perform penetration-testing against a target network. While initial results were promising, scalability was problematic, making this approach not feasible for real-work scenarios.

Recently, Pasquale et al. [45] proposed ChainReactor which uses the PDDL planning language to find multi-step exploitation chains in container setups. They used a prototype to fully enumerate a target system, translate the enumeration data into the PDDL language and applied manually written PDDL rules to find multi-step exploitation chains using an existing lifted PDDL solver. Their prototype was able to find two classes of vulnerabilities (*cron-jobs* and *systemd* unit files, both with wrong file permissions) which had to be exploited manually thus making this a non-autonomous system.

2.4.3 LLMs for Offensive Security. We want to give a chronologically ordered¹¹ overview of research into using LLMs for penetration-testing. An overview of the reviewed publications can be seen in Table 1.

Initial Forays. To the best of our knowledge, Happe and Cito [14] first investigated the use of autonomous LLMs for solving Linux privilege-escalation attacks. They wrote a prototype that uses a single-level LLM-driven control-loop to autonomously execute system commands on a connected Linux virtual machine containing security vulnerabilities and insecure configurations. Deng et al. [7] concurrently investigated the use of LLMs to interactively perform penetration tests against CTF machines. LLMs are used both to create high-level penetration-test plans (“Pentest-Task-Trees”, PTTs) as well as suggest concrete penetration-testing commands. The provided commands are then executed by a human operator who is allowed agency in fixing errors within the LLM-derived commands, i.e., is allowed to fix parameter errors.

Automated Single-Target Exploitation. In a follow-up paper, Happe et al. [18] further detail the usage of LLMs for privilege escalation by introducing a Linux privilege-escalation benchmark and investigating multiple LLM configurations, including a *Plan-and-Solve* setup. Fang et al. [10] investigated the capabilities of LLMs to initially hack web-sites and later extended their research into one- and zero-day development [9, 11]. Shao et al. [52, 52] used the NYU CTF benchmark to analyze capabilities of LLMs over a diverse range of tasks, ranging from cryptography challenges to web penetration-testing. Xu et al. [65] introduce a LLM-guided autonomous hacking tool that utilized MetaSploit to attack a target virtual machine. Hyuang et al. [22] integrate both offensive as well as defensive capabilities into *PenHeal*. Similar to Shao et al., multiple authors create CTF-like benchmarks and evaluate the efficacy of LLMs against those single-host machines (Zhang et al. [70], Gioacchini et al. [12], Isozaki et al. [23]). Wu et al. [63], Muzsai et al. [36], and

¹¹We use the arXiv initial submission date for creating our ordering.